NON-STATIC MEMBERS

The non-static members are class members declared directly in the class block. There are four types of non-static members in Java as follows:

1. Non - Static Variables
2. Non - Static Methods
3. Non - Static Initializers
4. Constructors (special non-static methods)

NON-STATIC VARIABLES:

The variables declared in global scope (class block) without static keyword are called non-static variables. Having no modifier changes the characteristics of the variable.

SYNTAX:

```
class ClassName
{
    datatype identifier;
}
```

Characteristics of a Non - Static Variable:

1. Not prefixed with 'static' modifier/keyword.
2. Declared in class block and hence has global scope.
3. Stored in Heap area.
4. Initialized with default values.
5. Accessed with object reference only
6. Any number of non-static variables can be declared.

Accessing a non-static variable:

```
1  package sampledemo;
2
3  class Sample
4  {
5      String a;
6      int b;
7      public static void main(String[] args)
8      {
9          Sample s = new Sample();
10         // s is object reference of Sample type
11         System.out.println(s.a);
12         System.out.println(s.b);
13     }
14 }
15
```

```
<terminated
null
0
```

NON-STATIC METHODS:

The methods which DO NOT have 'static' modifier are called non-static methods.

SYNTAX:

[access modifier] return_type method_name ([formal arguments])

{

}

Characteristics of a Non-Static method:

1. Must NOT have 'static' modifier.
2. Declared in class block like every method.
3. Storage of non-static method:
   - Method block along with statements in Method area
   - Signature and Reference in Heap area
4. Accessed with method call statement:
   - Object_reference.method_signature;
5. Any number of non-static methods can be declared.

Accessing a non-static method:

```java
package sampledemo;

class Sample
{
    public void abc()
    {
        System.out.println("non-static method");
    }
    public String xyz()
    {
        return "non-static string returning method";
    }
    public static void main(String[] args)
    {
        Sample s = new Sample();
        // s is object reference of Sample type
        s.abc();
        System.out.println(s.xyz());
    }
}
```

```
<terminated> Sample [Java Application] C:\Users\naray\.p2\pool\plu
non-static method
non-static string returning method
```

NON - STATIC INITIALIZERS:

Initializers in Java are used to execute start-up instructions. There are two types of Non - Static Initializers.

- Single Line Non - Static Initializers (SL NSI)
- Multi Line Non - Static Initializers (ML NSI)

Single Line Non-Static Initializers:

The declaration of a non-static variable along with initialization is called as SL NSI.

SYNTAX:

datatype identifier = value;

Characteristics of SL NSI:

1. They DO NOT have 'static' modifier and must be initialized with a value.
2. They are declared in the class block and hence global scope.
3. They are stored in the Heap area with the value initialized.
4. They can be accessed with object reference only
5. Any number of SL NSI can be declared.

Multi Line Non-Static Initializers:

un-named block to code multiple statements of start-up instructions.

SYNTAX:

```
{
    //statements;
}
```

Characteristics of ML NSI:

1. They DO NOT have 'static' modifier/keyword, only an un-named block.
2. They are declared in the class block and hence global scope.
3. Storage of ML NSI:
   The block along with statements in Method area
   Reference in Heap area (since no name)
4. Accessing ML NSI: They get executed implicitly and cannot be called
5. Any number of ML NSI can be declared

Speciality of ML NSI:

- They get executed even before the main method.
- ML NSI get implicitly executed once for each object created as they are non-static members.
- If there are more than one ML NSI, they get executed in top-down approach.
- If there are both MLSI and ML NSI, MLSI gets executed first.

SL NSI and ML NSI example:

```java
package sampledemo;

class Sample
{
    int a = 21;
    String b = "bbc";
    {
        System.out.println("ML non SI 1");
    }
    {
        System.out.println("ML non SI 2");
    }
    public static void main(String[] args)
    {
        Sample s = new Sample();
        // s is object reference of Sample type
        System.out.println(s.a);
        System.out.println(s.b);
    }
    static
    {
        System.out.println("MLSI");
    }
}
```

```
<terminated> Sample [Java App
MLSI
ML non SI 1
ML non SI 2
21
bbc
```

CONSTRUCTORS

These are special non-static members. They are methods with name same as ClassName (exactly same, including case) and called using Constructor calling statement.

Purpose of Constructors:

They are used to load all the non-static members of the class into the object that is created. They are also used to re-initialize the attributes of the object (default values with user passed values).

SYNTAX:

ClassName([formal arguments])

{

//re-initialization statements;

}

Why other terminologies of method syntax are not used?

A constructor does not have return type because, it acts like a LOADER to load the non-static members into the object and does not execute itself to return a value.

It is non-static, hence no ‘static’ modifier. Generally, treated as public access modifier.

TYPES OF CONSTRUCTORS:

There are two types of Constructors.

1. No Argument constructors
2. Parameterized constructors

No Argument constructors:

They have no formal arguments. They perform just one task, to load the non-static members of the class into the object created. They can be created in two ways:

1. Implicitly by the compiler – only when there is zero constructor in the class
2. Explicitly by the programmer

Since the no argument constructor is created implicitly by the compiler, it is also called as default constructor.

Parameterized constructors:

They must have formal arguments. They can load non-static members and re-initialize the attributes (non-static variables) of the class as well. For re-initializing, a keyword called 'this' is used.

'this' keyword:

- In-built non-static variable
- It is used to store the reference of the currently executing object

Let us create a class with attributes to understand the function of 'this' keyword.

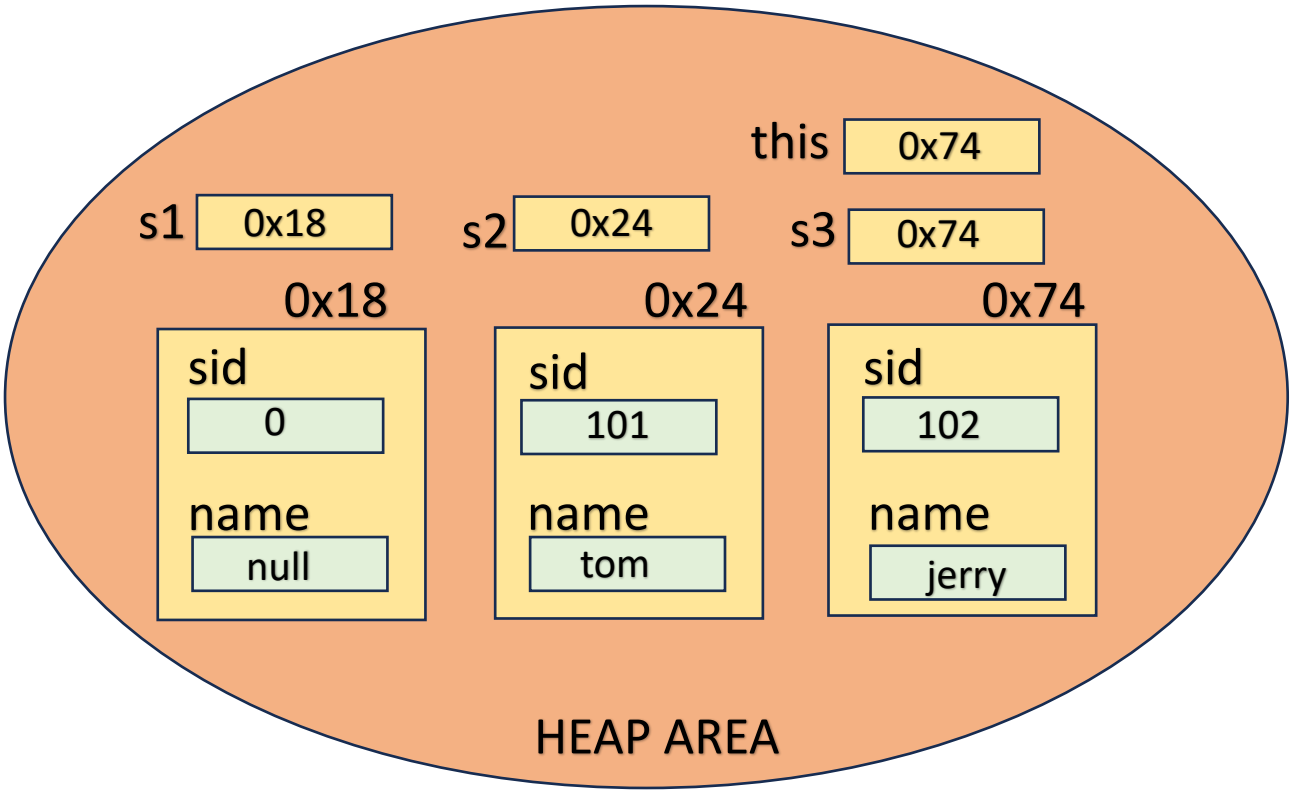
```
class Student
{
    int sid;
    String name;
    Student()
    {

    }
    Student(int sid, String name)
    {
        this.sid = sid;
        this.name = name;
    }
    public static void main(String[] args)
    {
        // s1,s2,s3 are object references of Student type
        Student s1 = new Student();
        System.out.println(s1.sid);
        System.out.println(s1.name);
        Student s2 = new Student();
        s2.sid = 101;
        s2.name = "tom";
        System.out.println(s2.sid);
        System.out.println(s2.name);
        Student s3 = new Student(102,"jerry");
        System.out.println(s3.sid);
        System.out.println(s3.name);
    }

}
```

```
<terminated> Student [Ja
0
null
101
tom
102
jerry
```

For the above example, Heap area memory allocation is as follows:



Parameterized constructors can be used to initialize all the attributes of the class in a single step, any number of times by creating multiple objects.

CONSTRUCTOR OVERLOADING:

When a class has more than one constructor with different formal arguments, it is called constructor overloading. All constructors will have same name as the ClassName. It is used to handle different combination of inputs from end user in an application.

RULES FOR CONSTRUCTOR OVERLOADING:

The signature of the constructor is considered only as datatype and not the attribute name.

Eg: Student (String name, String gender, int id) is taken as Student (String, String, int) by the compiler.

1. Two constructors cannot have one attribute of same type
   Eg: Student (String name) and Student (String gender) is NOT allowed
2. Two constructors cannot have attribute datatypes in the same order, even if attribute name is different

Eg: Student (String name, int id) and Student (String gender, int id) is NOT allowed

3. But it can be in different order
   Eg: Student (String name, int id) and Student (int id, String name) is allowed

CONSTRUCTOR CHAINING:

Calling a constructor from another constructor is called constructor chaining. It is of two types:

- Calling a constructor of the same class – **this() statement**
- Calling a constructor of different class (during inheritance) – **super() statement**

To call a constructor from another constructor, this() – 'this call statement' is used.

Characteristics of 'this call statement' – this() :

1. this() is used to call another constructor from a constructor
2. this() statement can be used only inside constructor body
3. It should always be the first statement - only one can be used
4. It calls the constructor of the same class
5. If there are 'n' constructors in a class, 'n-1' this() statements can be used
6. At least one constructor should be free of this() statement, because that constructor is used to load the non-static members into the object
7. Also, if this() statement is used in all constructors, it will lead to recursion.

WORKFLOW OF THIS CALL STATEMENT:

The workflow of this() statement is similar to method call statement.

- When this() statement is encountered, the execution of the current constructor will be paused and control goes to called constructor.
- After called constructor is executed, the execution of remaining code in calling constructor is resumed.

Example program for Constructor, Constructor Overloading and Constructor Chaining:

```java
package sampledemo;

class Student
{
    int sid;
    String name;
    String gender;
    Student()
    {
    }
    Student(int sid)
    {
        this.sid = sid;
    }
    Student(int sid, String name)
    {
        this(sid);
        this.name = name;
    }
//  Student(int sid, String gender) //ERROR - DUPLICATE METHOD
//  {
//
//  }
    Student(int sid, String name, String gender)
    {
        this(sid, name);
        this.gender = gender;
    }
    public static void main(String[] args)
    {
        // s is object references of Student type
        Student s = new Student(102,"jerry", "male");
        System.out.println(s.sid);
        System.out.println(s.name);
        System.out.println(s.gender);
    }

}
```

```
<terminated>
102
jerry
male
```

These are the four non-static members of a class in Java.